



Server-Side Web Development

(06016418)



พื นนจา (TA)



วันนี้เราจะได้เรียนอะไรกันบ้าง ?





Let us Talk About You

If you are someone with:

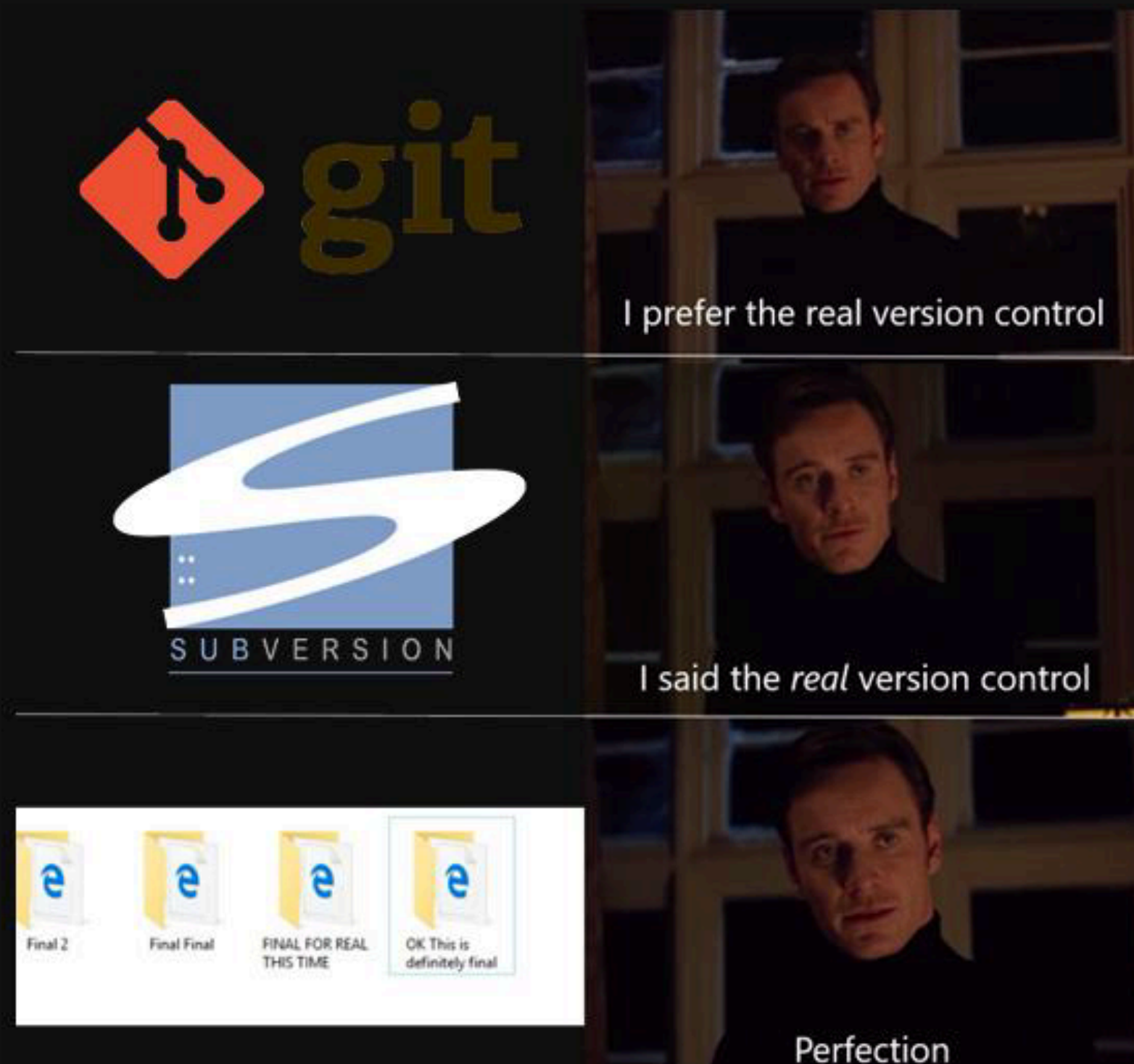
- Bachelor's Degree in Computer Science, IT or other related fields.
- 1 - 9 years+ experience in **Java**, **Golang**, **Kotiln** software development.
- Experience in Spring Boot or Spring MVC framework is a plus
- Strong problem solving skill, good attitude and teamwork.
- Preferred experience and knowledge in Web service development based on J2EE framework including Servlet, Java beans, EJB, JMS, JavaMail, Web Services, HTML, XML, UML etc.
- Using enterprise level database (e.g. Oracle, MSSQL) Eclipse, Netbeans or Jetbrain IDE **git** version control system
- Strong knowledge in OOP software and REST/SOAP web services design and implementation.

Job Specification:

- Bachelor's Degree or above in Computer Science, Computer Engineering, Information Technology or related field.
- At least 1 years experience in software development, programming, or coding (If new graduated, must be knowledgeable in related field).
- Skills in computer language and Framework
 - Node.js, React.js, GO, JAVA, Spring boot, TypeScript, Flutter, Kotlin
- Knowledge in AWS or any Cloud platform
- Experience with source control systems such as **Git**, SVN, etc.
- Can work under pressure, strong communication & negotiation skills, self-learning, self-motivated.
- Experience in Kafka, Redis, API Gateway are beneficial
- Strong Experience in modern application development by using microservices approach



Version Control System คืออะไร ?



Version Control :

เป็นระบบจัดเก็บประวัติการเปลี่ยนแปลงของไฟล์ ใช้ได้กับไฟล์เกือบทุกชนิด
เช่น โค้ด, รูปภาพ, เอกสาร (แต่หลัก ๆ จะเก็บแค่ Code)
เหมาะสำหรับทั้งงานเดี่ยวและทีม (ในทีมคือใช้แน่ ๆ)
ย้อนกลับไฟล์ไปยังเวอร์ชันก่อนหน้า (ย้อนโค้ดกลับไปตอนที่มันพังได้)
ดูว่าใครเปลี่ยนอะไร, เมื่อไร

คำศัพท์เทคนิค :

Version Control = ระบบควบคุมเวอร์ชัน

Revert = การย้อนกลับไปยังสถานะก่อนหน้า



ระบบ Version Control แบบ Local

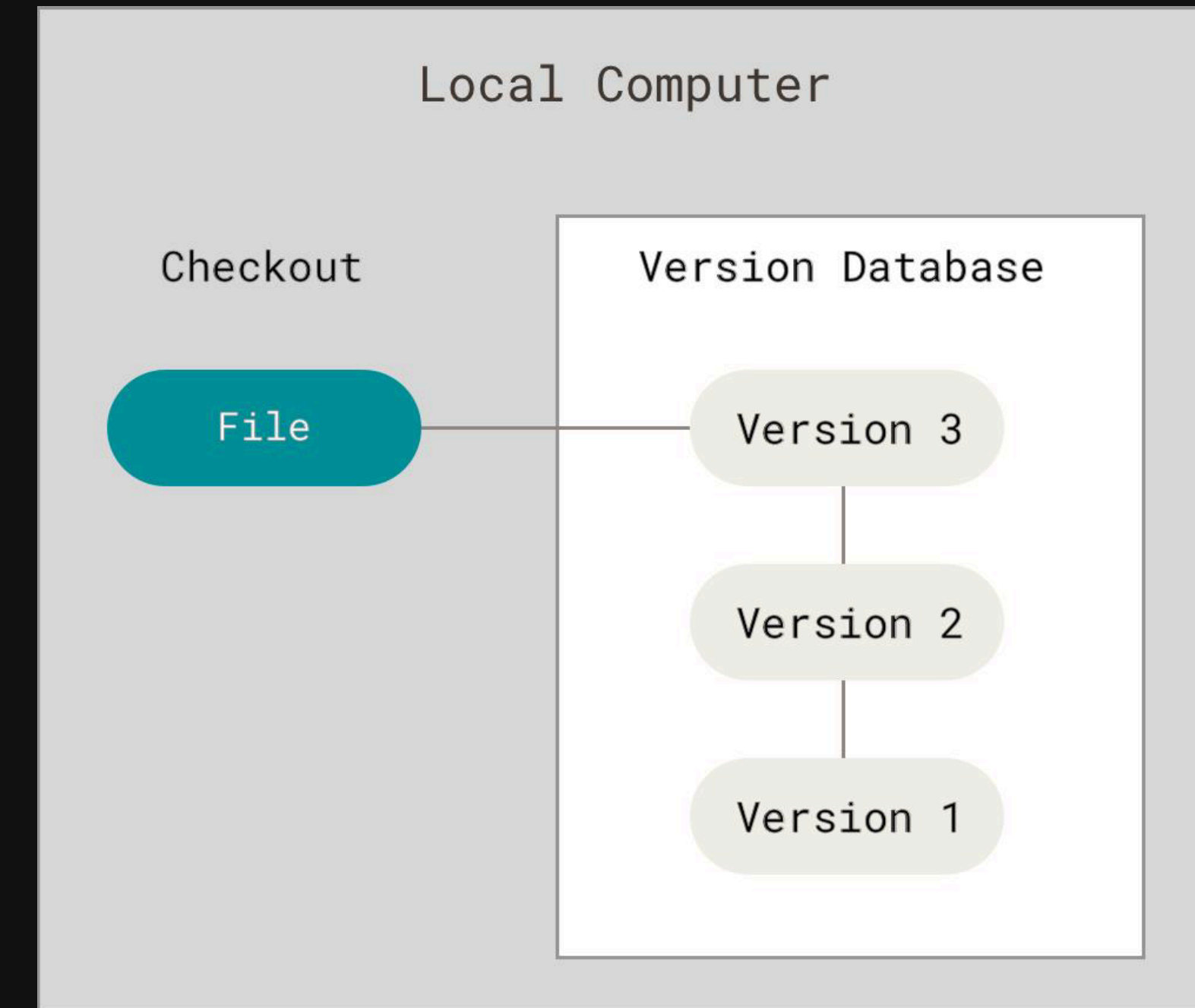
วิธีดั้งเดิม : ก๊อปปี้ไฟล์ไว้ในโฟลเดอร์ใหม่แบบ Manual ปัญหาคือ
สับสน, เขียนทับไปแล้วไม่รู้จะกลับมาแก้ยังไง

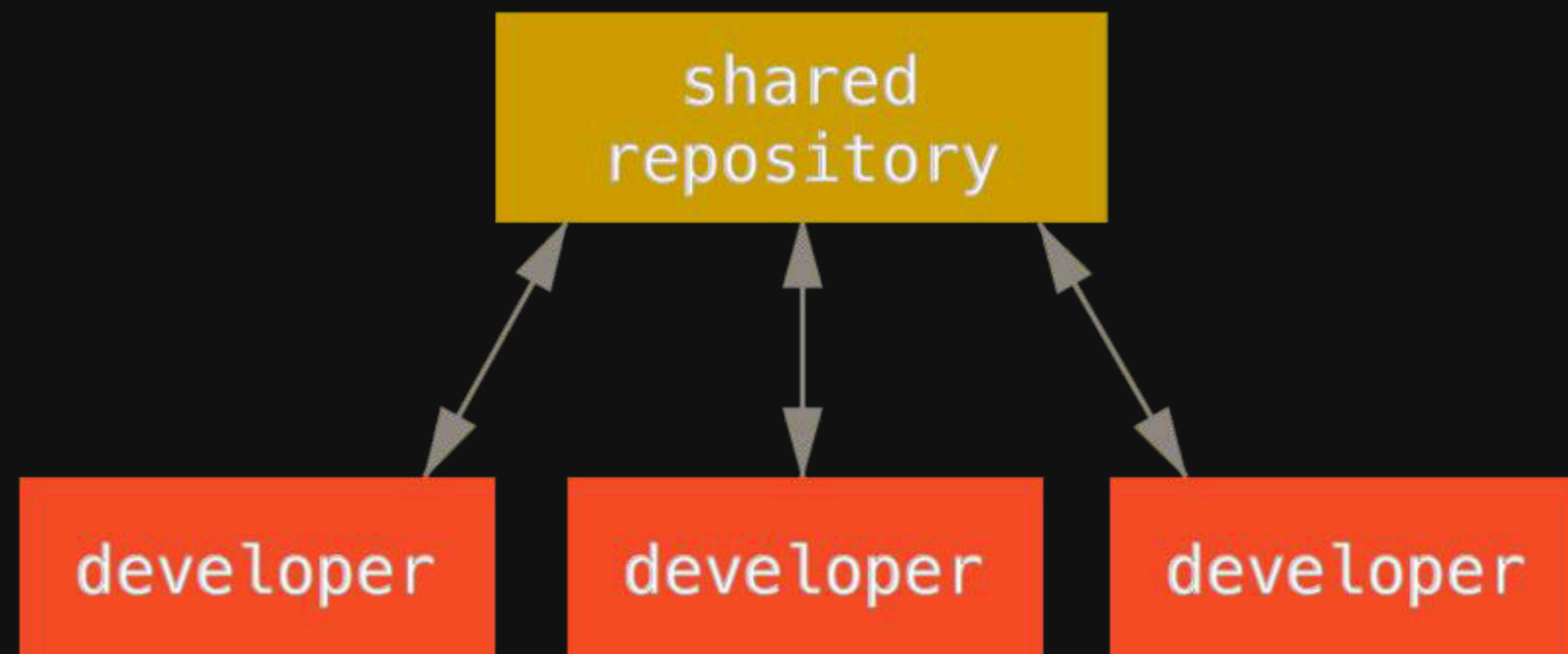
แนวคิดของ Local VCS

ใช้ database เก็บ patch ของไฟล์

คำศัพท์เทคนิค :

Patch = ความแตกต่างระหว่างไฟล์เวอร์ชันหนึ่งกับอีกเวอร์ชัน





Centralized Version Control (CVCS)

Server กลาง + Client หลายเครื่อง

ตัวอย่าง : CVS, Subversion (SVN), Perforce

ข้อดี : ทุกคนเห็นการเปลี่ยนแปลงของกันและกัน จัดการสิทธิ์การเข้าถึงได้ง่าย

ข้อเสีย : จุดเสี่ยงเดียว (Single Point of Failure)

ถ้า Server พัง = ทุกอย่างหยุดทำงาน

คำศัพท์เทคนิค :

Single Point of Failure = จุดเดียวที่ถ้าพังจะกระทบทั้งระบบ



Distributed Version Control (DVCS)

ทุกเครื่องที่ clone = มี repository พร้อม history คน

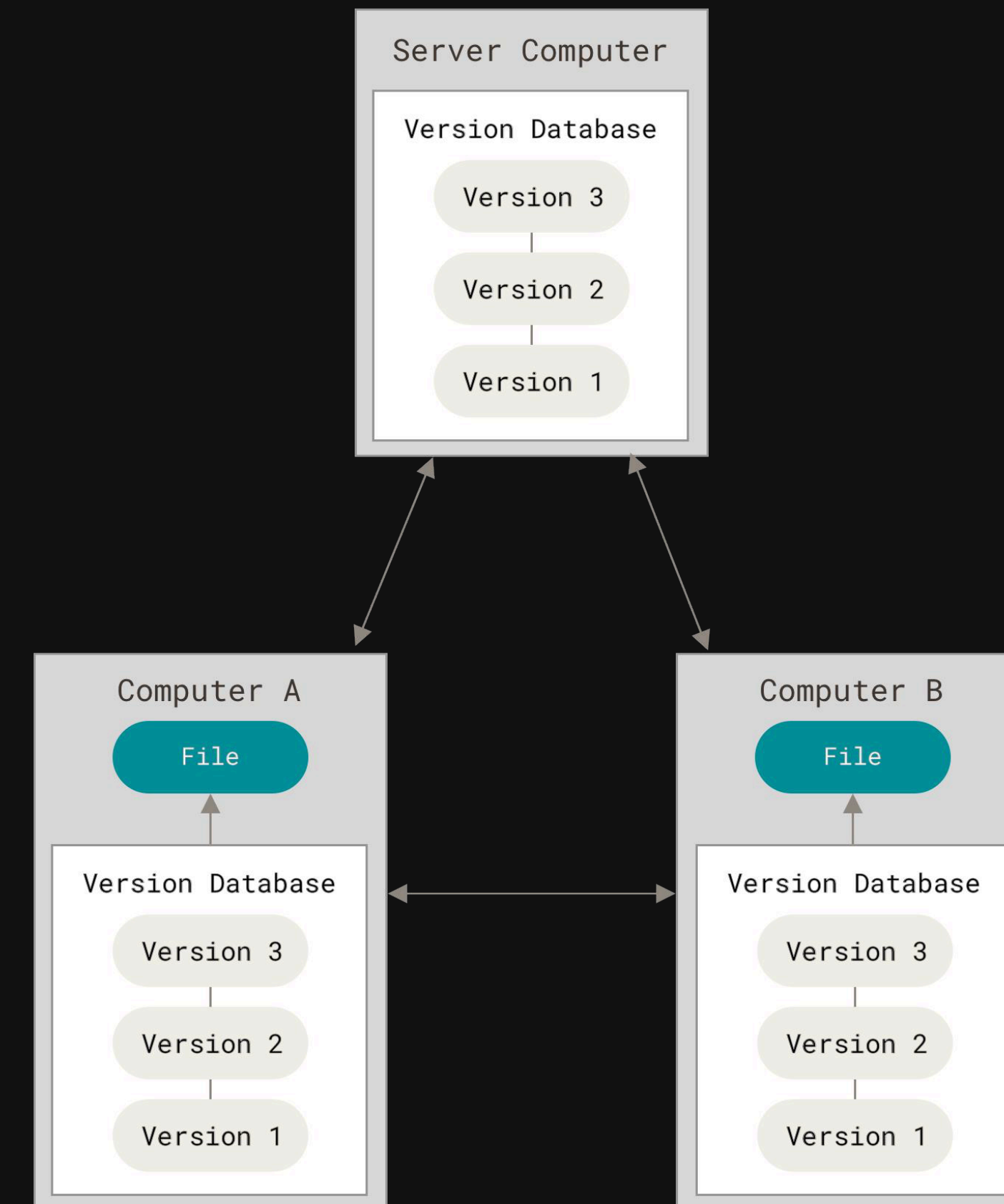
ตัวอย่าง : Git, Mercurial, Darcs

ข้อดี : ไม่ต้องพึ่ง server กลาง

คำศัพท์เทคนิค :

Clone = สำเนา Repository ทั้งหมด

Repository = ที่เก็บเวอร์ชันของไฟล์และประวัติการเปลี่ยนแปลง





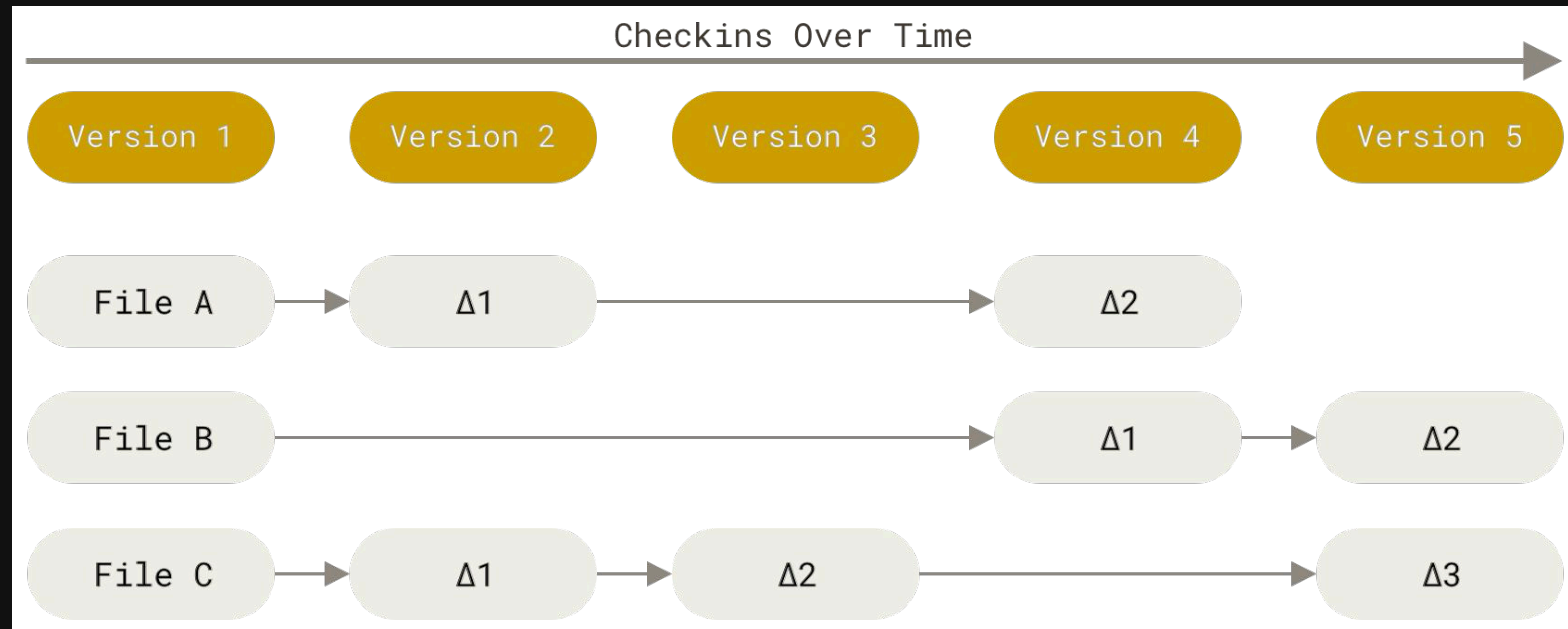
The Creator of Git and Linux



Linus Torvalds decides to build "his own new tool" -> Git



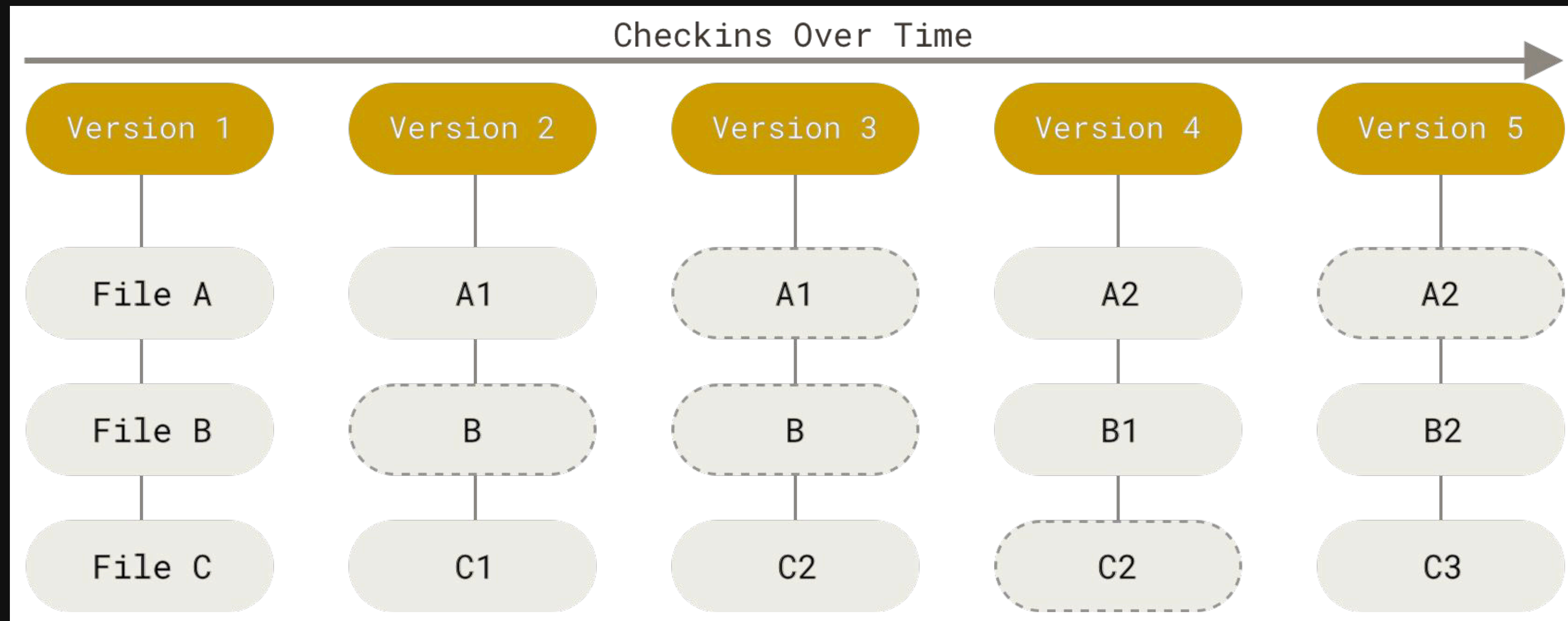
Git เก็บข้อมูลเป็นภาพรวม (Snapshots) ไม่ใช่แค่ความแตกต่าง (Deltas)



delta-based



Git เก็บข้อมูลเป็นภาพรวม (Snapshots) ไม่ใช่แค่ความแตกต่าง (Deltas)



snapshot-based



Git มีระบบตรวจสอบความถูกต้องในตัว

The mechanism that Git uses for this checksumming is called a SHA-1 hash. This is a 40-character string composed of hexadecimal characters (0–9 and a–f) and calculated based on the contents of a file or directory structure in Git. A SHA-1 hash looks something like this:

```
24b9da6552252987aa493b52f8696cd6d3b00373
```

ทุกอย่างถูก checksum ด้วย SHA-1 ก่อนเก็บ

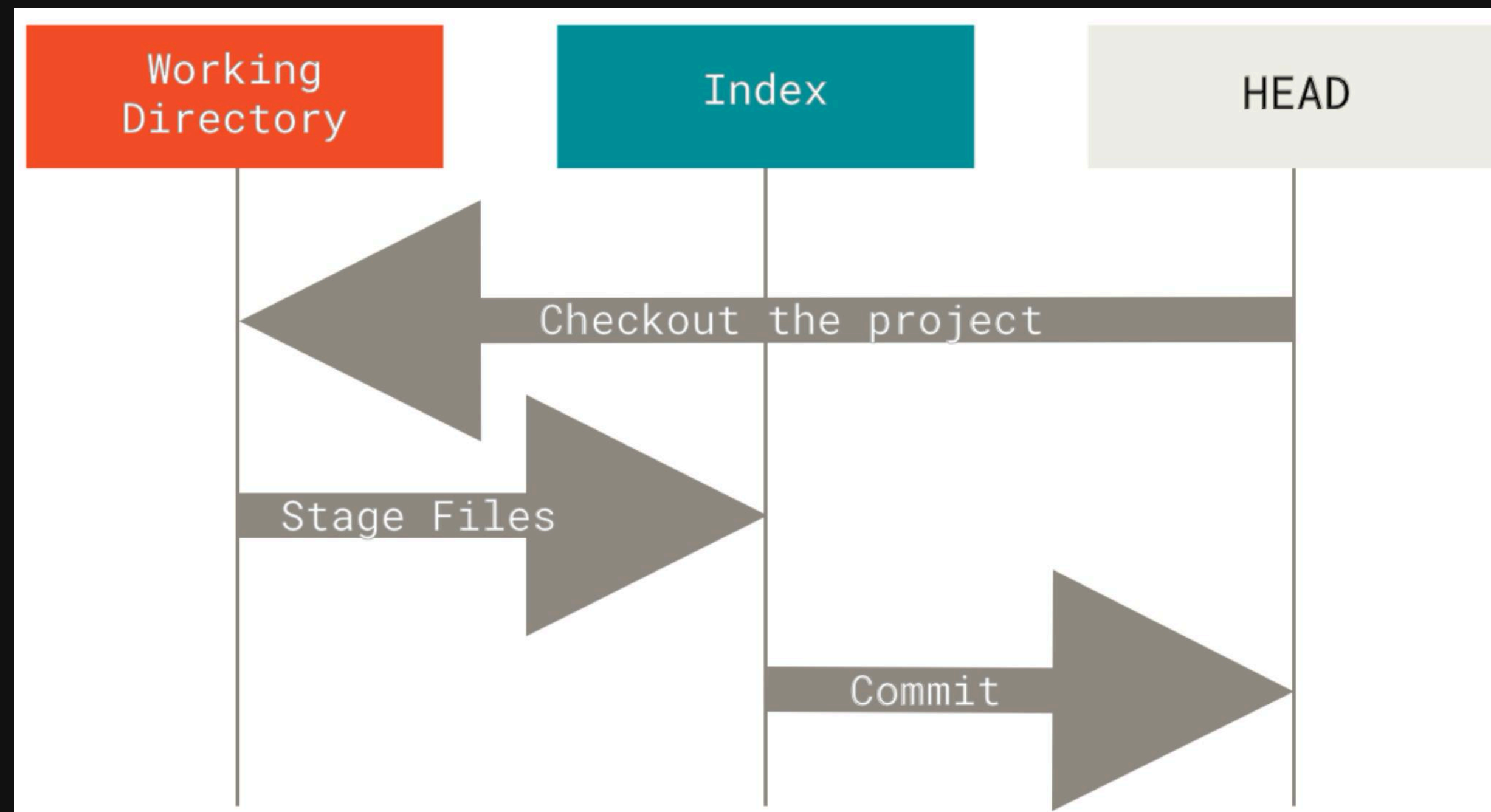
Git ไม่เก็บข้อมูลโดยอิงจากชื่อไฟล์

แต่เก็บตาม hash แทนไม่มีคำสั่งไหนที่ลบข้อมูลโดยตรง

สามารถย้อนกลับได้เกือบทุกการกระทำ (ถ้า commit แล้ว)



3 ส่วนหลักของ Git



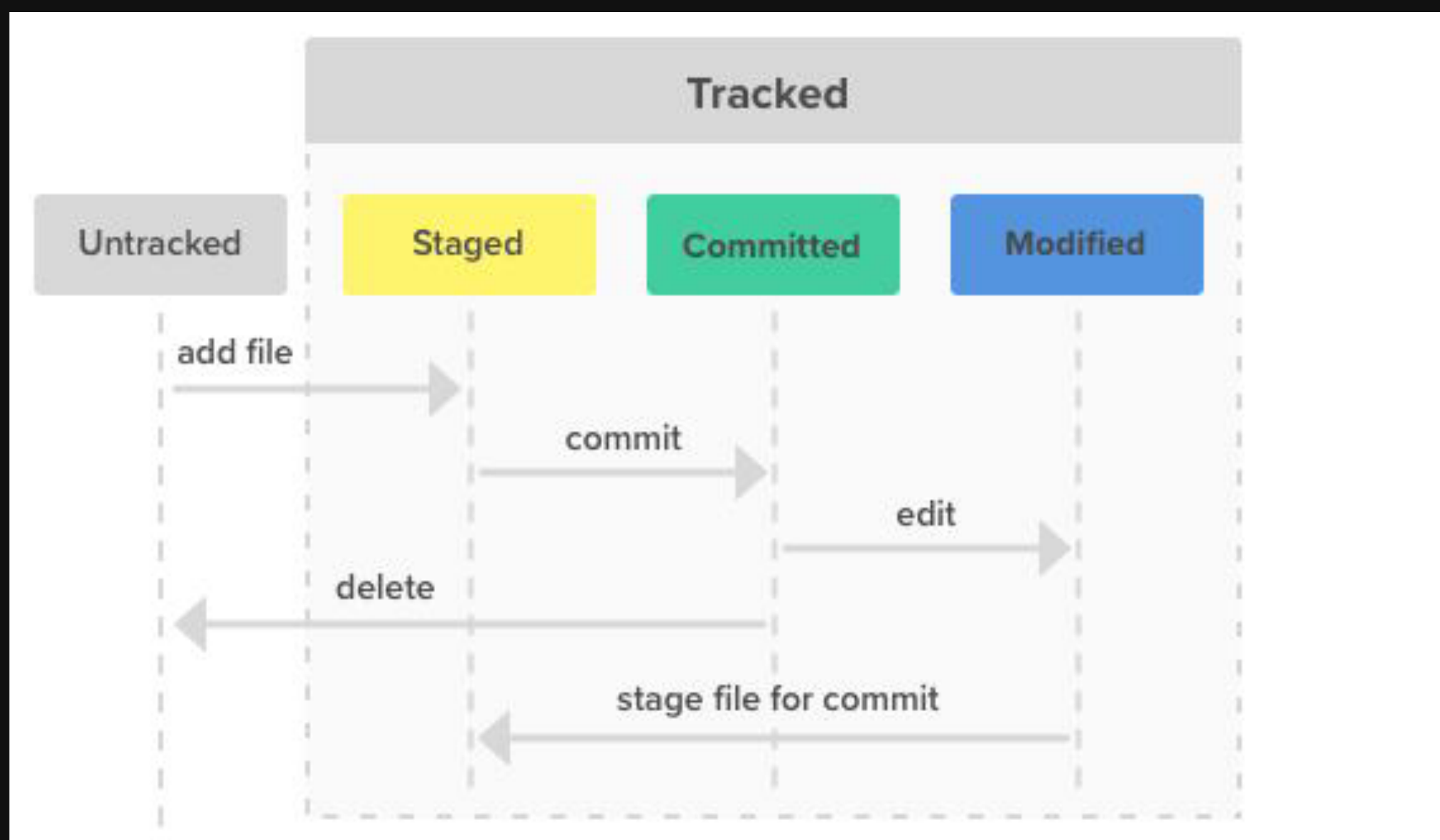
Working Tree = โฟลเดอร์ที่เราใช้งานจริง

Staging Area (หรือ Index) = พื้นที่เก็บไฟล์ที่รอ commit

Git Directory = ที่เก็บ database และ metadata ทั้งหมด



สถานะ ของ Git



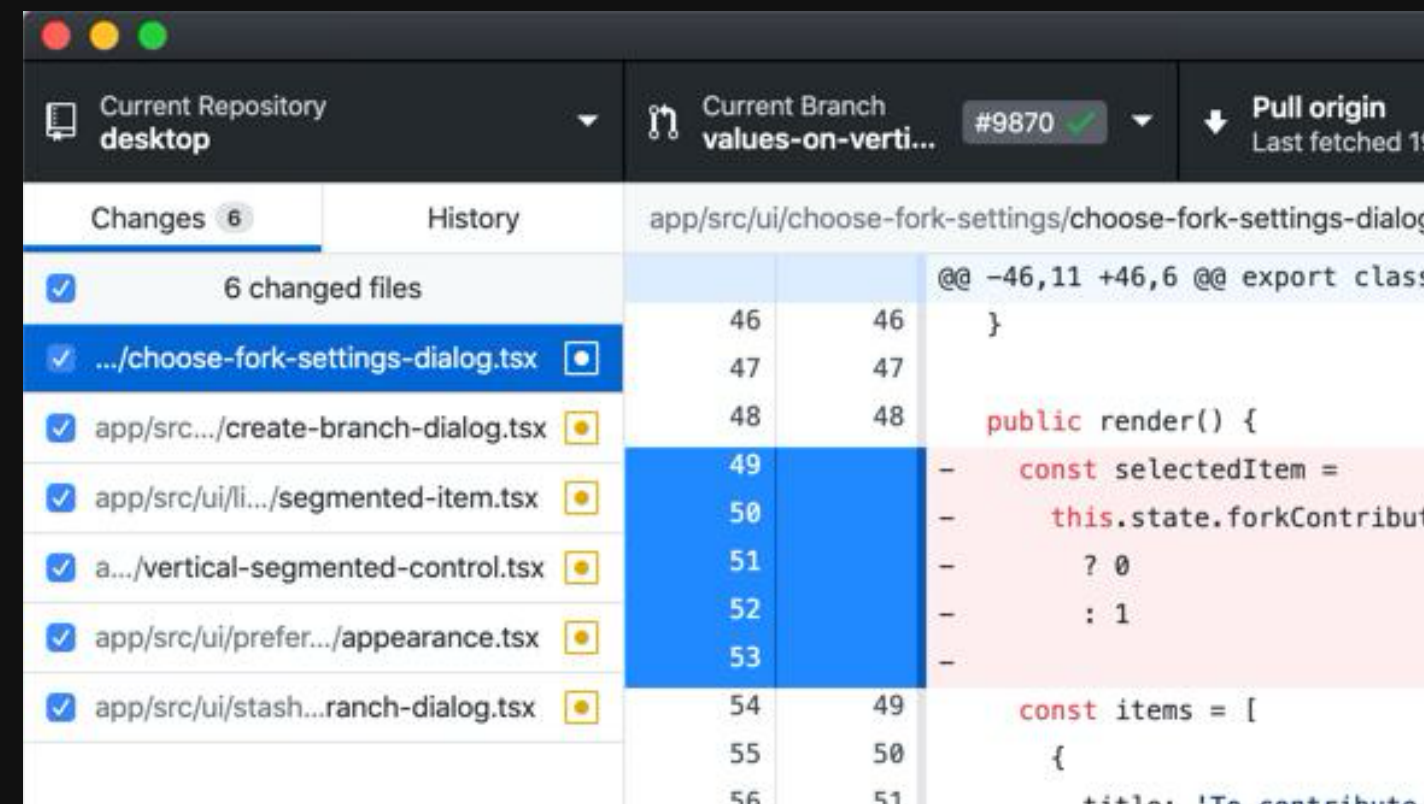
Modified : แก้ไขแล้ว แต่ยังไม่ได้เตรียม commit

Staged : ไฟล์ที่เลือกแล้วว่าจะ commit

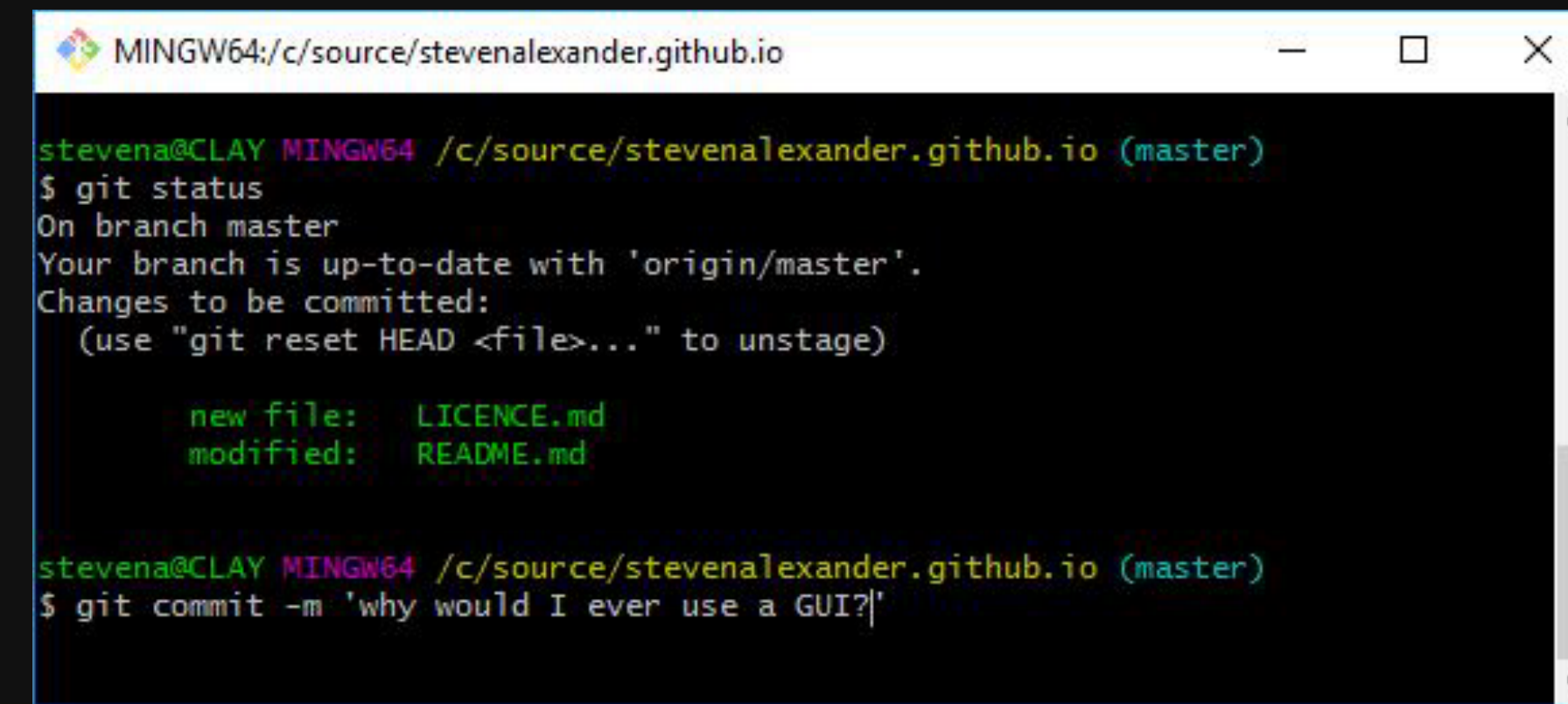
Committed : ข้อมูลถูกบันทึกใน Git อย่างถาวร



การใช้งาน Git



ผ่าน GUI (กราฟิก หน้าต่าง)



ผ่าน Command Line



ทำไมต้องใช้ Git ผ่าน Command Line ?

ครอบคลุม ทุกคำสั่งของ Git

GUI ส่วนใหญ่รองรับเพียงบางคำสั่งเท่านั้น

หากใช้ Command Line เป็น -> เรียนรู้ GUI ได้ง่ายขึ้น



🔥 Git CLI vs GitHub Desktop

การดู 4 พัน ครั้ง • 2 ปีที่แล้ว

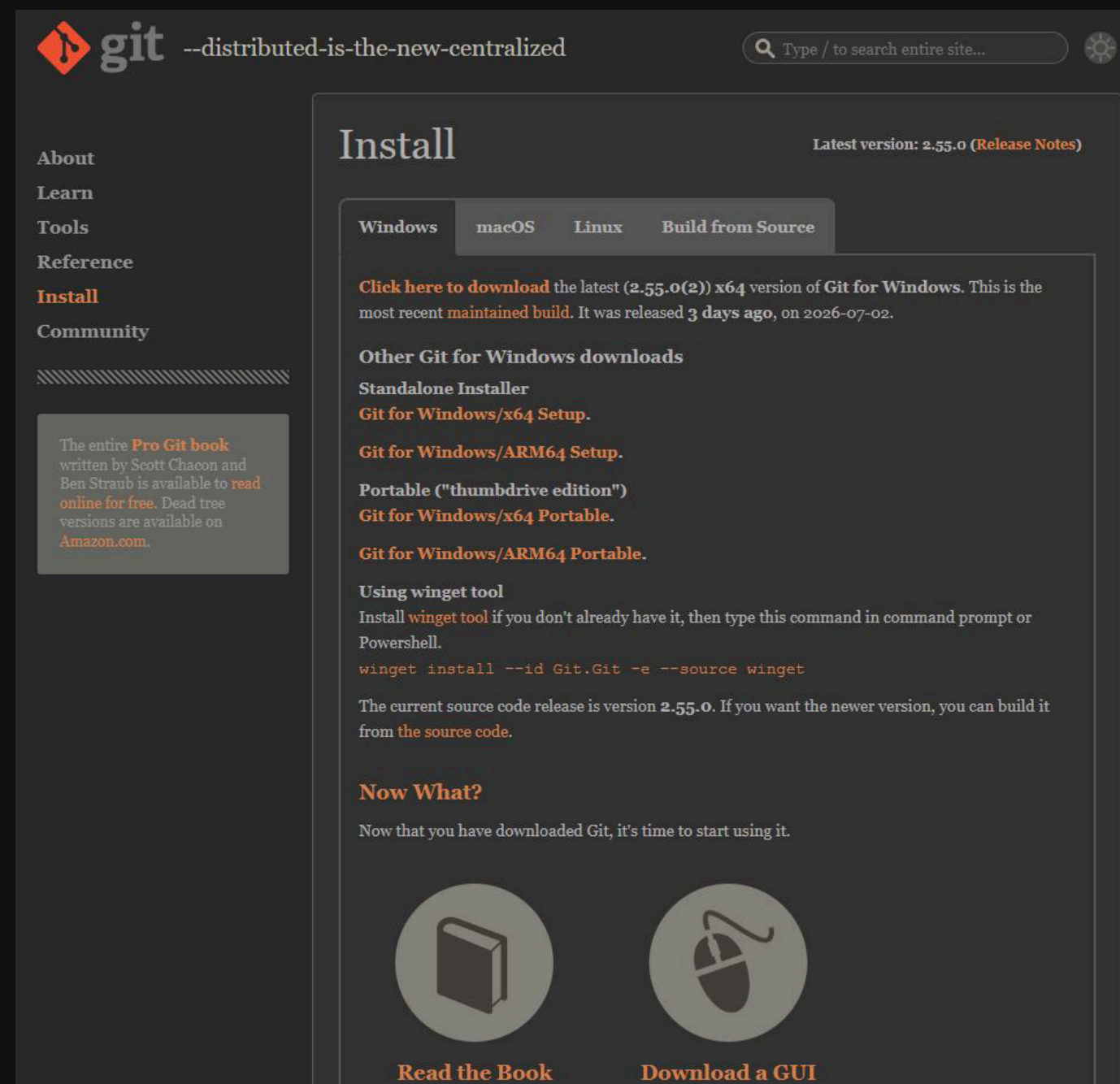
BorntoDev

Git CLI และ GitHub Desktop ต่างเป็นเครื่องมือสำหรับการทำงานกับ Git และ GitHub repositories แล้วเคยสงสัยกันไหมว่า 2 ...

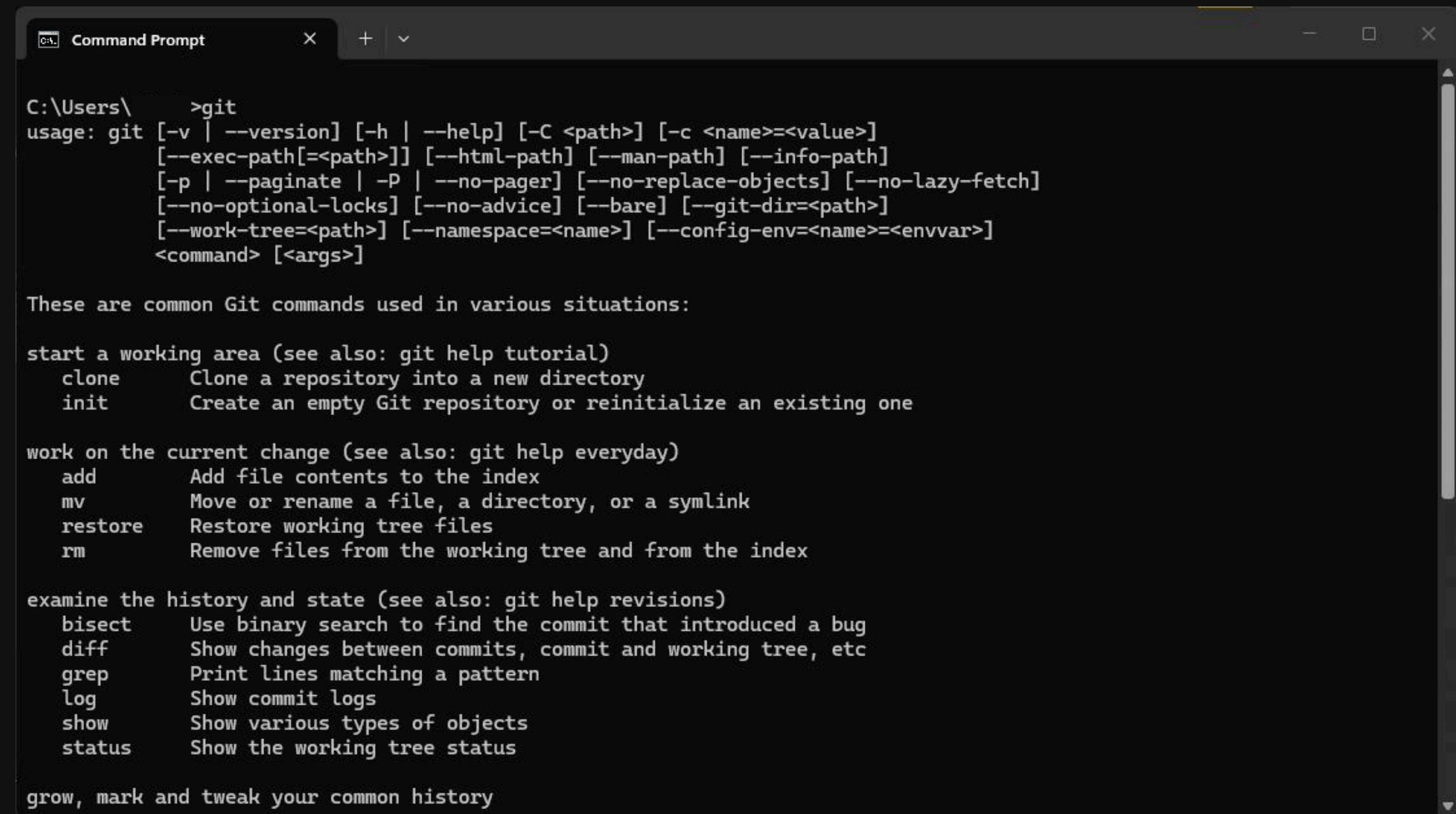
<https://youtu.be/oxoIr7TyGMs?si=2SqgAHxpzpAfj0bS>



ติดตั้ง Git



<https://git-scm.com/install/windows>





การตั้งค่า Git ครั้งแรก

ใช้คำสั่ง git config เพื่อตั้งค่าต่าง ๆ



การตั้งค่ามี 3 ระดับ :

System -> ใช้กับทุก user บนเครื่อง

Global -> ใช้กับ user ปัจจุบัน (แนะนำสำหรับผู้เริ่มต้น)

Local -> ใช้เฉพาะใน repository นั้น ๆ



คำสั่งดูค่าที่ตั้งไว้



```
git config --list --show-origin
```

```
Command Prompt
C:\Users\ >git config --list --show-origin
file:C:/Program Files/Git/etc/gitconfig diff.astextplain.textconv=astextplain
file:C:/Program Files/Git/etc/gitconfig filter.lfs.clean=git-lfs clean -- %f
file:C:/Program Files/Git/etc/gitconfig filter.lfs.smudge=git-lfs smudge -- %f
file:C:/Program Files/Git/etc/gitconfig filter.lfs.process=git-lfs filter-process
file:C:/Program Files/Git/etc/gitconfig filter.lfs.required=true
file:C:/Program Files/Git/etc/gitconfig http.sslbackend=openssl
file:C:/Program Files/Git/etc/gitconfig http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
file:C:/Program Files/Git/etc/gitconfig core.autocrlf=true
file:C:/Program Files/Git/etc/gitconfig core.fscache=true
file:C:/Program Files/Git/etc/gitconfig core.symlinks=false
file:C:/Program Files/Git/etc/gitconfig pull.rebase=false
file:C:/Program Files/Git/etc/gitconfig credential.helper=manager
file:C:/Program Files/Git/etc/gitconfig credential.https://dev.azure.com.usehttppath=true
file:C:/Program Files/Git/etc/gitconfig init.defaultbranch=master
file:C:/Users/Ninja/.gitconfig user.email=66070010@kmitl.ac.th
file:C:/Users/Ninja/.gitconfig user.name=66070010Ninja
file:C:/Users/Ninja/.gitconfig filter.lfs.clean=git-lfs clean -- %f
file:C:/Users/Ninja/.gitconfig filter.lfs.smudge=git-lfs smudge -- %f
file:C:/Users/Ninja/.gitconfig filter.lfs.process=git-lfs filter-process
file:C:/Users/Ninja/.gitconfig filter.lfs.required=true
file:.git/config core.repositoryformatversion=0
file:.git/config core.filemode=false
file:.git/config core.bare=false
file:.git/config core.logallrefupdates=true
file:.git/config core.symlinks=false
file:.git/config core.ignorecase=true
file:.git/config remote.origin.url=https://github.com/DiscoveryPiscine42Bkk/fun-with-coding-feb-2025-66070010Ninja.git
```



ตั้งค่า Identity, Editor และ Branch เริ่มต้น



```
git config --global user.name "John Don"  
git config --global user.email johndon@gmail.com
```

ถ้าต้องการดูเฉพาะค่าใดค่าหนึ่ง



```
git config user.name
```

```
Command Prompt  
C:\Users\Ninja>git config user.name  
C:\Users\Ninja>
```




ดูคู่มือแบบเต็ม (manpage)

```
git help
```

```
Command Prompt
C:\Users\ >git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
[--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
[-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
[--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
[--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
<command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect    Use binary search to find the commit that introduced a bug
  diff      Show changes between commits, commit and working tree, etc
  grep      Print lines matching a pattern
  log       Show commit logs
  show      Show various types of objects
  status    Show the working tree status

grow, mark and tweak your common history
  backfill  Download missing objects in a partial clone
```

ดูแบบสรุป (quick help)

```
git add -h
```

```
Command Prompt
C:\Users\ >git add -h
usage: git add [<options>] [--] <paths>...

-n, --[no-]dry-run      dry run
-v, --[no-]verbose      be verbose

-i, --[no-]interactive
                        interactive picking
-p, --[no-]patch        select hunks interactively
--[no-]auto-advance     auto advance to the next file when selecting hunks interactively
-U, --unified <n>       generate diffs with <n> lines context
--inter-hunk-context <n>
                        show context between diff hunks up to the specified number of lines
-e, --[no-]edit         edit current diff and apply
-f, --[no-]force        allow adding otherwise ignored files
-u, --[no-]update       update tracked files
--[no-]renormalize      renormalize EOL of tracked files (implies -u)
-N, --[no-]intent-to-add
                        record only the fact that the path will be added later
-A, --[no-]all          add changes from all tracked and untracked files
--[no-]ignore-removal   ignore paths removed in the working tree (same as --no-all)
--[no-]refresh          don't add, only refresh the index
--[no-]ignore-errors    just skip files which cannot be added because of errors
--[no-]ignore-missing   check if - even missing - files are ignored in dry run
--[no-]sparse           allow updating entries outside of the sparse-checkout cone
--[no-]chmod (+|-)x     override the executable bit of the listed files
--[no-]pathspec-from-file <file>
                        read pathspec from file
--[no-]pathspec-file-nul
                        with --pathspec-from-file, pathspec elements are separated with NUL character
```



2 วิธีหลักในการเริ่มต้นใช้งาน Git

1. สร้าง Git repository จากโฟลเดอร์ที่มีอยู่ (git init)
2. Clone โปรเจกต์จาก Git repository ที่มีอยู่แล้ว (git clone)



```
md server-side-day2  
  
cd server-side-day2  
  
git init
```

```
Command Prompt  
C:\Users\Ninja\Desktop>md server-side-day2  
C:\Users\Ninja\Desktop>cd server-side-day2  
C:\Users\Ninja\Desktop\server-side-day2>git init  
Initialized empty Git repository in C:/Users/Ninja/Desktop/server-side-day2/.git/  
C:\Users\Ninja\Desktop\server-side-day2>
```

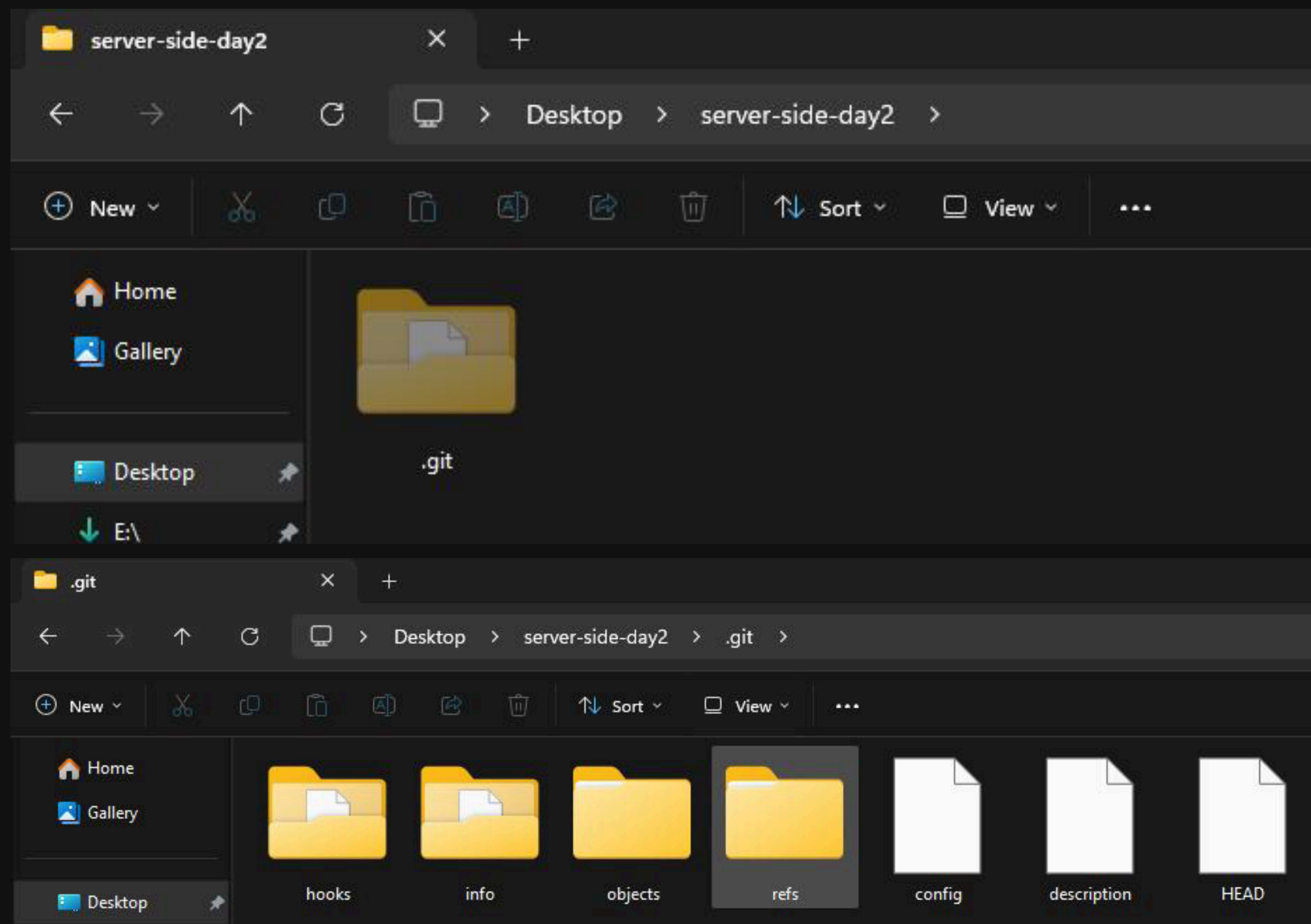



สร้าง Git repository จากโฟลเดอร์ที่มีอยู่ (git init)

```
Command Prompt
C:\Users\ \Desktop>md server-side-day2
C:\Users\ \Desktop>cd server-side-day2
C:\Users\ \Desktop\server-side-day2>git init
Initialized empty Git repository in C:/Users/Ninja/Desktop/server-side-day2/.git/
C:\Users\ \Desktop\server-side-day2>
```



สร้าง Git repository จากโฟลเดอร์ที่มีอยู่ (git init)



hooks/ = เก็บสคริปต์ที่ Git จะรันอัตโนมัติเมื่อเกิดเหตุการณ์ต่าง ๆ เช่น commit หรือ push

info/ = ใช้เก็บไฟล์ exclude สำหรับ ignore ไฟล์แบบเฉพาะเครื่อง

objects/ = เก็บข้อมูลทั้งหมดของโปรเจกต์ในรูปแบบเข้ารหัส (commit, tree, blob)

refs/ = เก็บตำแหน่งของ branch และ tag ที่ใช้ในโปรเจกต์

config = ตั้งค่าการทำงานของ Git เฉพาะใน repository นี้

description = คำอธิบายสั้น ๆ ของ repo (ใช้ในบางระบบเท่านั้น)

HEAD = บอกว่าเรากำลังอยู่ที่ branch หรือ commit ใดในขณะนี้



สร้างไฟล์ใหม่ขึ้นมาก่อน

```
demo-http-server.js

const http = require('http');
const hostname = 'localhost';
const port = 3000;
const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

```
demo-http-server.js U X
demo-http-server.js > ...
1  const http = require('http');
2  const hostname = 'localhost';
3  const port = 3000;
4  const server = http.createServer((req, res) => {
5    res.statusCode = 200;
6    res.setHeader('Content-Type', 'text/plain');
7    res.end('Hello, World!\n');
8  });
9
10 server.listen(port, hostname, () => {
11   console.log(`Server running at http://${hostname}:${port}/`);
12 });
```

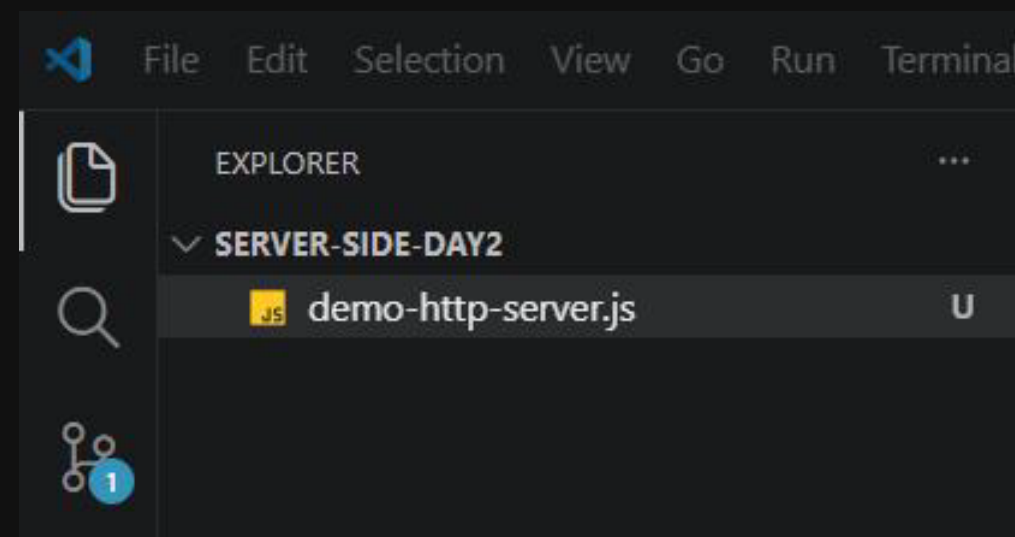
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ \Desktop\server-side-day2>node .\demo-http-server.js
Server running at http://localhost:3000/
█

localhost:3000 X
http://localhost:3000/
Hello, World!

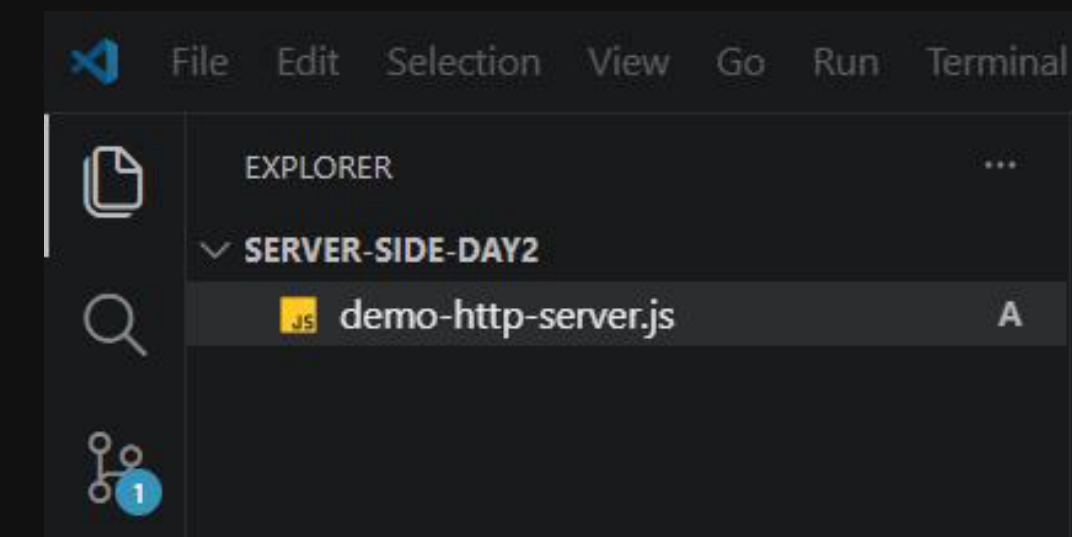


เริ่มติดตามไฟล์และ commit



git add = เพิ่มไฟล์เข้าสู่ staging area

```
git add .\demo-http-server.js
```



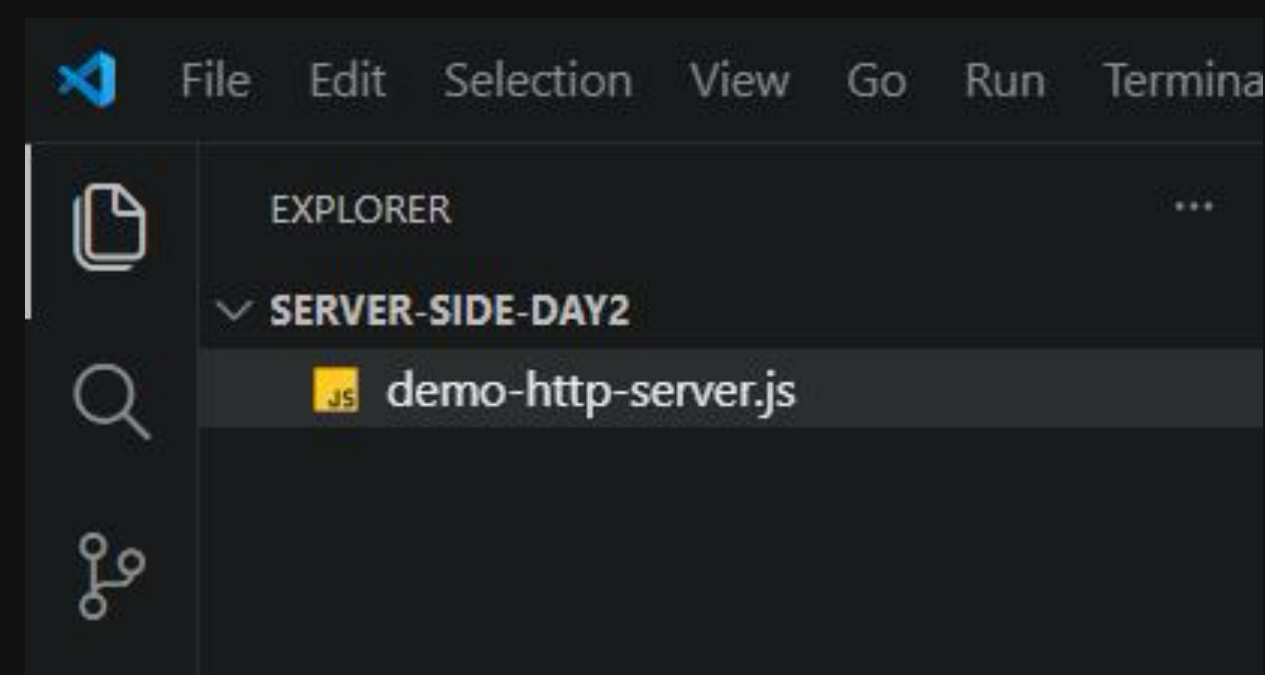


เริ่มติดตามไฟล์และ commit



```
git commit -m "Initial project version"
```

```
Command Prompt
C:\Users\ \Desktop\server-side-day2>git add .\demo-http-server.js
C:\Users\ \Desktop\server-side-day2>git commit -m "Initial project version"
[master (root-commit) 888e9fe] Initial project version
1 file changed, 12 insertions(+)
create mode 100644 demo-http-server.js
C:\Users\ \Desktop\server-side-day2>
```



git commit = บันทึกสถานะไฟล์ลง Git repository



ดู commit ที่เคยทำไปแล้ว



```
git log  
git log --oneline --graph --all
```

```
Command Prompt  
C:\Users\Ninja top\server-side-day2>git log  
commit 888e9fe19bf7b79a0605b64903a0afd814f86129 (HEAD -> master)  
Author: Ninja <66070010@kmitl.ac.th>  
Date: Sun Jul 5 21:44:33 2026 +0700  
  
Initial project version  
  
C:\Users\Ninja top\server-side-day2>git log --oneline --graph --all  
* 888e9fe (HEAD -> master) Initial project version  
  
C:\Users\Ninja top\server-side-day2>
```

git log = ทั่วไปจะแสดงประวัติการ Commit แบบละเอียด

git log --oneline --graph --all = ถ้าอยากดูแบบย่อและเป็น branch ด้วย



ลองแก้ไขแล้ว commit เพิ่ม

```
demo-http-server.js

const http = require('http');
const hostname = 'localhost';
const port = 3000;
const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

  if (req.url === '/about') {
    res.end('About Page\n');
  } else if (req.url === '/contact') {
    res.end('Contact Page\n');
  } else {
    res.end('Hello, World!\n');
  }
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

```
File Edit Selection View Go Run Terminal Help
server-side-day2

demo-http-server.js M X
demo-http-server.js > ...
You, now | 1 author (You)
1  const http = require('http');
2  const hostname = 'localhost';
3  const port = 3000;
4  const server = http.createServer((req, res) => {
5    res.statusCode = 200;
6    res.setHeader('Content-Type', 'text/plain');
7
8    if (req.url === '/about') {
9      res.end('About Page\n');
10   } else if (req.url === '/contact') {
11     res.end('Contact Page\n');
12   } else {
13     res.end('Hello, World!\n');
14   }
15 });
16
17 server.listen(port, hostname, () => {
18   console.log(`Server running at http://${hostname}:${port}/`);
19 });
```

เพิ่ม endpoint **/about** และ **/contact** แบบง่าย ๆ



บันทึกไฟล์ แล้วดูสถานะ



git status

```
Command Prompt
C:\Users\ \Desktop\server-side-day2>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   demo-http-server.js

no changes added to commit (use "git add" and/or "git commit -a")
C:\Users\ \Desktop\server-side-day2>
```

git status = บอกว่ามีไฟล์ไหนถูกแก้ไข และยังไม่ถูก stage ไว้สำหรับ commit



เพิ่มไฟล์เข้าไปใน Stage

```
Command Prompt
C:\Users\... \Desktop\server-side-day2>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   demo-http-server.js

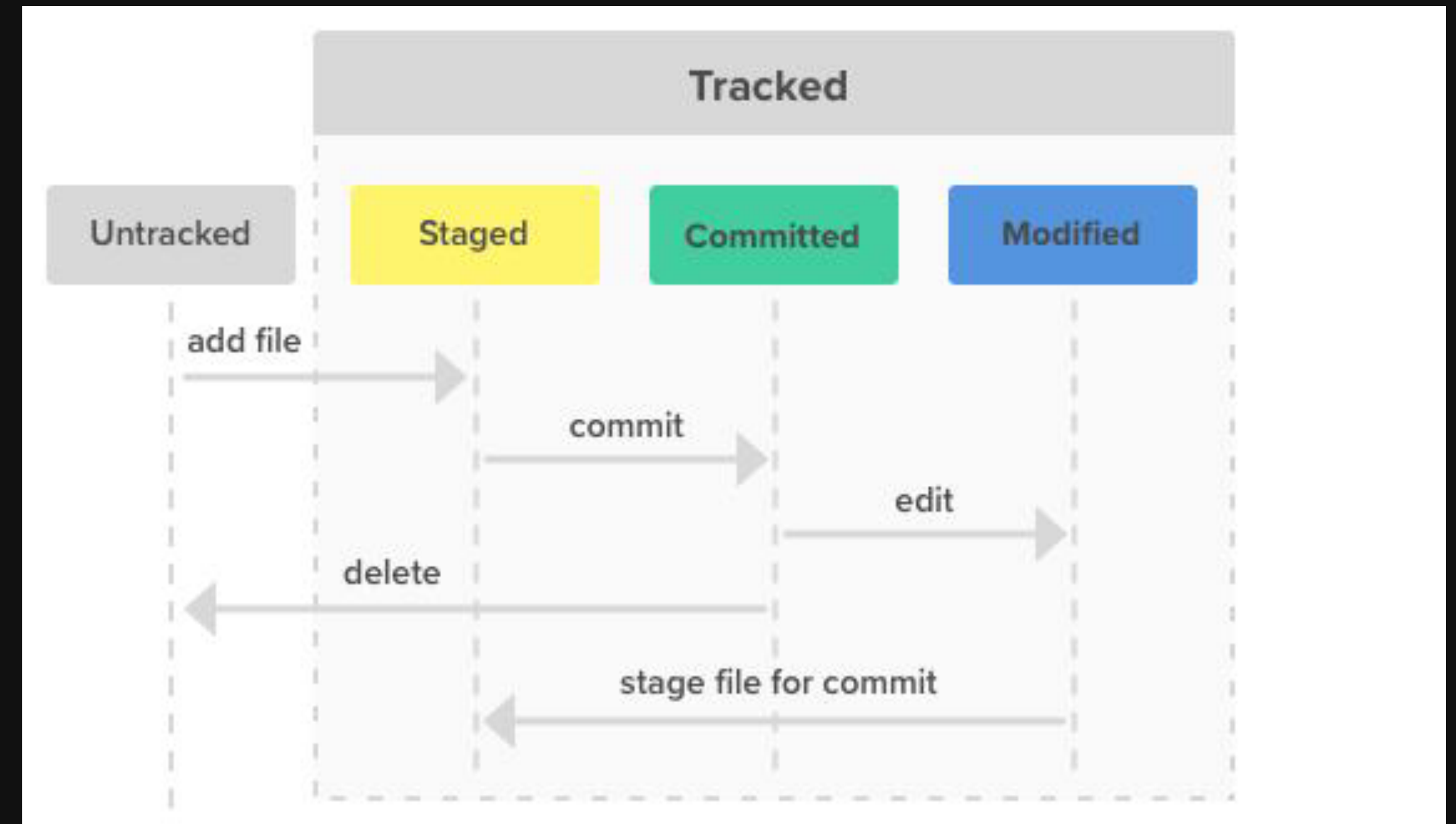
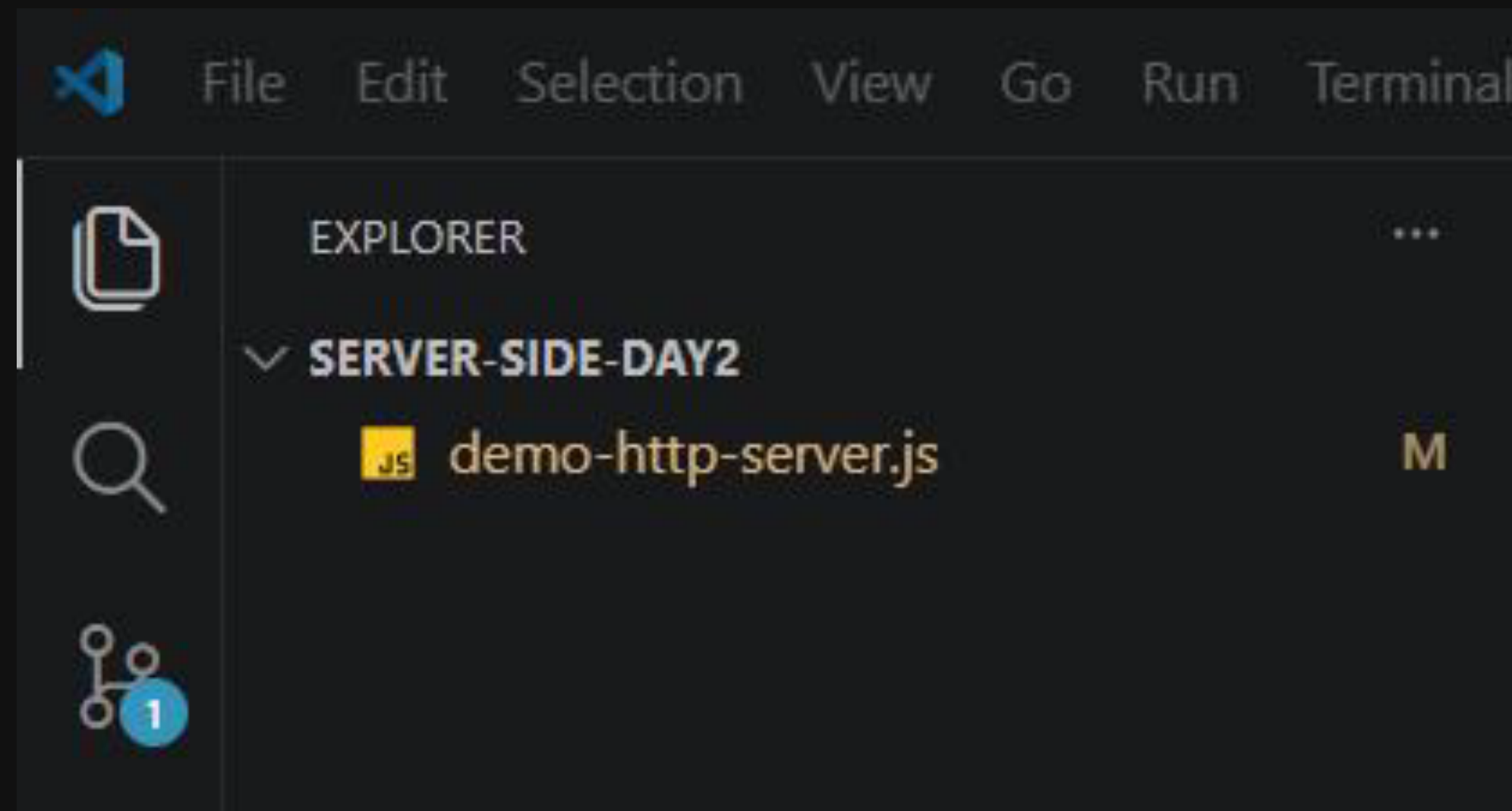
no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\... \Desktop\server-side-day2>git add .\demo-http-server.js

C:\Users\... \Desktop\server-side-day2>
```

รอบนี้จะใช้ git add ชื่อไฟล์เหมือนเดิมหรือ **git add .** เพื่อเพิ่มเข้าไปทั้งหมดก็ได้

จะเห็นว่าสถานะเป็น Modified





Commit การเปลี่ยนแปลง

```
git commit -m "Add /about and /contact routes"
```

```
C:\Users\Ninja\Desktop\server-side-day2>git commit -m "Add /about and /contact routes"
[master d39ffb5] Add /about and /contact routes
1 file changed, 8 insertions(+), 1 deletion(-)

C:\Users\Ninja\Desktop\server-side-day2>
```

```
C:\Users\      \Desktop\server-side-day2>git log --oneline
d39ffb5 (HEAD -> master) Add /about and /contact routes
888e9fe Initial project version

C:\Users\      \Desktop\server-side-day2>
```

เช็คประวัติอีกครั้งด้วย `git log --oneline`



สรุปคำสั่ง Git

`git log` → ดูประวัติ commit

`git status` → ตรวจสอบไฟล์ที่แก้ไข

`git add <file>` → เพิ่มไฟล์เข้าสู่ staging

`git status` → ตรวจสอบไฟล์ที่แก้ไข

`git commit -m “ข้อความ”` → บันทึก snapshot ใหม่

`git log --oneline` → ดูประวัติแบบสั้น



```
demo-http-server.js M X
demo-http-server.js > server > http.createServer() callback
...
1  const http = require('http');
2
3  const hostname = 'localhost';
4  const port = 3000;
5
6  const server = http.createServer((req, res) => {
7    if (req.url === '/about') {
8      res.statusCode = 200;
9      res.setHeader('Content-Type', 'text/plain');
10     res.end('About Page\n');
11   } else if (req.url === '/contact') {
12     res.statusCode = 200;
13     res.setHeader('Content-Type', 'text/plain');
14     res.end('Contact Page\n');
15   } else if (req.url === '/api') {
16     res.statusCode = 200;
17     res.setHeader('Content-Type', 'application/json');
18     const data = {
19       message: 'Hello from API',
20       timestamp: new Date().toISOString()
21     };
22   } else {
23     res.statusCode = 200;
24     res.end('Hello, World!\n');
25   }
26 });
27
28 server.listen(port, hostname, () => {
29   console.log(`Server running at http://${hostname}:${port}/`);
30 });
```

ปรับให้โค้ดเพิ่มน้อย เพื่อดูรายละเอียดการ
เปลี่ยนแปลงจากคำสั่ง **git diff**

```
Command Prompt - git diff
diff --git a/demo-http-server.js b/demo-http-server.js
index 625569f..15676ce 100644
--- a/demo-http-server.js
+++ b/demo-http-server.js
@@ -1,15 +1,26 @@
 const http = require('http');
+
+ const hostname = 'localhost';
+ const port = 3000;
-const server = http.createServer((req, res) => {
-  res.statusCode = 200;
-  res.setHeader('Content-Type', 'text/plain');
+const server = http.createServer((req, res) => {
+  if (req.url === '/about') {
+    res.end('About Page\n');
+    res.statusCode = 200;
+    res.setHeader('Content-Type', 'text/plain');
+    res.end('About Page\n');
+  } else if (req.url === '/contact') {
+    res.statusCode = 200;
+    res.setHeader('Content-Type', 'text/plain');
+    res.end('Contact Page\n');
+  } else if (req.url === '/api') {
+    res.statusCode = 200;
+    res.setHeader('Content-Type', 'application/json');
+    const data = {
+      message: 'Hello from API',
+      timestamp: new Date().toISOString()
+    };
+  } else {
+    res.statusCode = 200;
+    res.end('Hello, World!\n');
+  }
+});
```




ทำกระบวนการเดิมซ้ำ

1. เพิ่มการเปลี่ยนแปลง ด้วย git add ชื่อไฟล์
2. Commit
3. ดูประวัติ

```
Command Prompt
C:\Users\ \Desktop\server-side-day2>git add .\demo-http-server.js

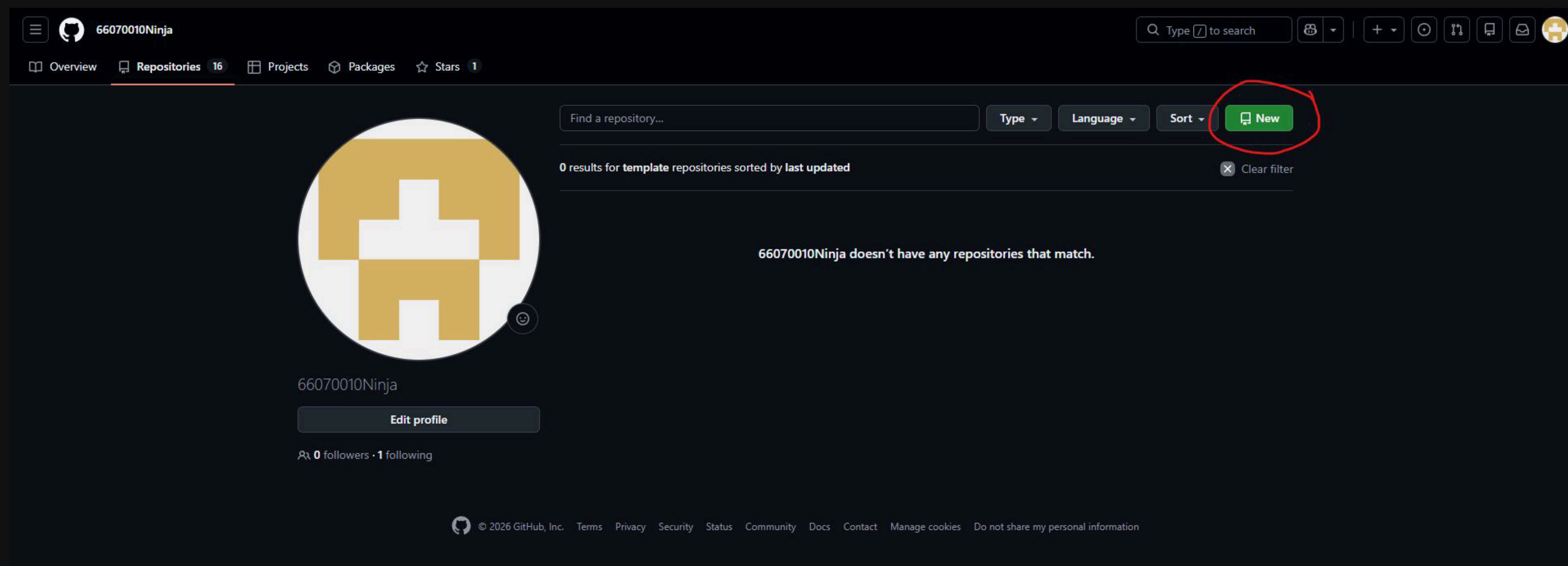
C:\Users\ \Desktop\server-side-day2>git commit -m "Add /api route with JSON response"
[master a6b0542] Add /api route with JSON response
1 file changed, 15 insertions(+), 4 deletions(-)

C:\Users\ \Desktop\server-side-day2>git log --oneline
a6b0542 (HEAD -> master) Add /api route with JSON response
d39ffb5 Add /about and /contact routes
888e9fe Initial project version

C:\Users\ \Desktop\server-side-day2>
```



การสร้าง Git repository



ไปที่หน้า "Repositories" บน Github > คลิก "New"



ได้ Repository แรกแล้ว !

my_repo Public

Pin Watch 0 Fork 0 Star 0

**Start coding with Codespaces**
Add a README file and start coding in a secure, configurable, and dedicated development environment.
[Create a codespace](#)

**Add collaborators to this repository**
Search for people using their GitHub username or email address.
[Invite collaborators](#)

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH

https://github.com/66070010Ninja/my_repo.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# my_repo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/66070010Ninja/my_repo.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/66070010Ninja/my_repo.git
git branch -M main
git push -u origin main
```

 **ProTip!** Use the URL for this page when adding GitHub as a remote.



ขั้นตอนในการเชื่อม Git

สำหรับการเชื่อม Git ในเครื่องกับ remote repository "**ครั้งแรก**" มีขั้นตอนดังต่อไปนี้

- เปิด Terminal หรือ Command Prompt
- ไปยังโฟลเดอร์ที่เก็บไฟล์โค้ด
- รันคำสั่ง **git init** เพื่อเริ่มต้น Git repository ในโฟลเดอร์
- เพิ่มไฟล์โค้ดเข้าสู่ staging area ด้วยคำสั่ง **git add <ชื่อไฟล์>**
- commit การเปลี่ยนแปลงด้วยคำสั่ง **git commit -m "<ข้อความอธิบายการเปลี่ยนแปลง>"**

```
C:\Users\Ninja\Desktop\my-first-repo>git init
Initialized empty Git repository in C:/Users/Ninja/Desktop/my-first-repo/.git/

C:\Users\Ninja\Desktop\my-first-repo>git add .\app.js

C:\Users\Ninja\Desktop\my-first-repo>git commit -m "feat: Add Hello World message"
[master (root-commit) 559e993] feat: Add Hello World message
1 file changed, 1 insertion(+)
create mode 100644 app.js

C:\Users\Ninja\Desktop\my-first-repo>
```


ขั้นตอนในการเชื่อม GIT (2)

สำหรับการเชื่อม Git ในเครื่องกับ remote repository “ครั้งแรก” มีขั้นตอนดังต่อไปนี้

```

Command Prompt
C:\Users\Ninja\Desktop\my-first-repo>git remote add origin https://github.com/66070010Ninja/my_repo.git

C:\Users\Ninja\Desktop\my-first-repo>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 246 bytes | 246.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/66070010Ninja/my_repo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\Ninja\Desktop\my-first-repo>

```

- กำหนดว่าจะโยนขึ้นไป branch ไหน เช่น `git branch -M main` จะไปที่ main
- บอกให้ Git รู้ว่าจะไปที่ Remote repository ไหน เช่น `git remote add origin <URL ของ Remote repository>`
- พร้อมแล้วก็อัปโหลดโค้ดด้วยคำสั่ง `git push -u origin main`



FINAL RESULT

แค่นี้โค้ดของเราก็จะไปอยู่บน GitHub เรียบร้อยแล้ว

